



**Instituto Tecnológico de Costa Rica**

Escuela de Ingeniería en Computadores

Análisis Numérico para la Ingeniería - CE1111

**Informe Problema 2**

Profesor: Juan Pablo Soto Quirós

Alumnos:

Mauro Andrés Navarro Obando

Paula Melina Porras Mora

Grupo: 1

1 de junio 2026

---

# 1. Pregunta 2: Cota de error del polinomio de interpolación

## 1.1. Estrategia utilizada

Para resolver la pregunta se implementó una función en Python llamada `cota_interpolacion(f, a, b, xv, xb)`. La función recibe la función  $f$ , el intervalo de análisis  $[a, b]$ , el vector de nodos de interpolación  $xv$  y el punto  $x_b$  donde se desea estimar la cota del error.

Con estos datos, se calcula el grado del polinomio interpolador como:

$$n = \text{cantidad de nodos} - 1$$

Luego, se aplica la fórmula teórica de la cota de error del polinomio de interpolación. Para ello, se necesita calcular el producto entre las diferencias  $(x_b - x_i)$  y una aproximación del máximo de la derivada de orden  $n + 1$  de la función  $f$  en el intervalo dado.

## 1.2. Fórmula matemática utilizada

Si  $p_n(x)$  es el polinomio interpolador de grado  $n$ , construido a partir de los nodos  $x_0, x_1, \dots, x_n$ , entonces la cota del error en un punto  $x_b$  se expresa como:

$$|f(x_b) - p_n(x_b)| \leq \frac{M}{(n+1)!} |(x_b - x_0)(x_b - x_1) \cdots (x_b - x_n)|$$

donde  $M$  representa el máximo valor de la derivada de orden  $n + 1$  de la función  $f$  en el intervalo  $[a, b]$ :

$$M = \max_{x \in [a, b]} |f^{(n+1)}(x)|$$

En este caso, como se usan 8 nodos, el polinomio interpolador es de grado:

$$n = 8 - 1 = 7$$

Por lo tanto, se utiliza la derivada de orden:

$$n + 1 = 8$$

## 1.3. Cálculo computacional del máximo de la derivada

El valor de  $M$  se obtuvo de forma computacional. Primero, se usó SymPy para calcular simbólicamente la derivada de orden 8 de la función  $f$ . Después, esa derivada se convirtió en una función numérica usando `lambdify`. Finalmente, se evaluó el valor absoluto de dicha derivada en una malla uniforme de puntos dentro del intervalo  $[0, 1, 0, 8]$ .

El mayor valor encontrado en esa malla se tomó como una aproximación de  $M$ . Esta estrategia no busca escribir manualmente la derivada de orden 8, ya que sería muy extensa, sino calcularla directamente con herramientas simbólicas y evaluarla numéricamente. En la ejecución realizada, el máximo aproximado se encontró cerca del extremo derecho del intervalo, es decir, en  $x = 0,8$ .

## 1.4. Datos utilizados de la Pregunta 1

Para aplicar la función `cota_interpolacion`, se utilizaron los mismos datos definidos en la Pregunta 1. La función analizada fue:

$$f(x) = \frac{\ln(\arcsin(x))}{\ln(x)}$$

El intervalo de análisis considerado fue:

$$[0,1,0,8]$$

Además, se usó el siguiente vector de nodos de interpolación:

$$xv = [0,1, 0,2, 0,3, 0,4, 0,5, 0,6, 0,7, 0,8]$$

Como el vector contiene 8 nodos, el polinomio interpolador construido es de grado:

$$n = 7$$

Por esta razón, para aplicar la fórmula de la cota de error se utilizó la derivada de orden:

$$n + 1 = 8$$

Finalmente, el punto donde se desea calcular la cota del error fue:

$$x_b = 0,55$$

## 1.5. Resultados obtenidos

Los resultados obtenidos mediante la implementación computacional se muestran en la Figura [1](#).

```

--- Pregunta 2: Cota de error del polinomio de interpolacion ---
Funcion f utilizada:
f(x) = ln(asin(x)) / ln(x)

Intervalo de analisis: [0.1, 0.8]
Vector de nodos de interpolacion xv:
[0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8]
Grado del polinomio interpolador: n = 7
Orden de la derivada utilizada: n + 1 = 8
Punto de evaluacion: xb = 0.55

Maximo aproximado de |f^(n+1)(x)| en [a,b]:
M ≈ 2.932341287356e+10
Este maximo se encontro aproximadamente en x = 0.800000

Producto |(xb-x0)(xb-x1)...(xb-xn)|:
|producto| ≈ 5.537109375000e-07

Cota de error obtenida:
ct ≈ 4.026957944672e-01
ct ≈ 0.402695794467

```

Figura 1: Resultados obtenidos para la cota de error del polinomio de interpolación.

A partir de los resultados mostrados, se obtuvo que:

$$M \approx 2,932341287356 \times 10^{10}$$

Este máximo aproximado se encontró en:

$$x \approx 0,8$$

El producto de las diferencias fue:

$$|(x_b - x_0)(x_b - x_1) \cdots (x_b - x_n)| \approx 5,537109375000 \times 10^{-7}$$

Por lo tanto, la cota de error obtenida fue:

$$ct \approx 4,026957944672 \times 10^{-1}$$

Es decir:

$$ct \approx 0,4027$$

## 1.6. Interpretación del resultado

La cota obtenida indica que, bajo la aproximación computacional realizada para  $M$ , el error absoluto entre la función original  $f$  y el polinomio interpolador  $p_7$  en el punto  $x_b = 0,55$  no debería superar aproximadamente 0,4027.

Este valor es una cota superior teórica, por lo que no necesariamente coincide con el error real. Normalmente, esta cota puede ser más grande que el error efectivo. Sin embargo, permite estimar un límite de seguridad para el error de interpolación usando los nodos definidos en la Pregunta 1.

## 1.7. Fragmento central de la implementación

La parte principal del cálculo de la cota en Python se resume en el siguiente procedimiento:

```
n = len(xv) - 1
orden = n + 1
M = max |f^(orden)(x)| en [a,b]
producto = abs((xb-x0)(xb-x1)...(xb-xn))
ct = (M / factorial(orden)) * producto
```